Structure and Interpretation of Computer Programs
Fall 2024, Nanjing University
Midterm test

注意事项（请仔细阅读）：

1. 试题的难度不是线性递增的，**注意时间分配**。在这项测验中，可能有些题目较难，因此你不要在一道题上思考时间太久，遇到不会答的题目可先跳过，如果有时间再去思考。否则，你可能没有时间完成后面的题目。
2. 代码填空题的**正确答案不止一种**。代码填空题答案正确即可得分，错误也不会倒扣分。每空**最多只能填一个** Python 语句或表达式，可以直接使用的 Python 内置函数（Python Built-in Functions）包括：`all`, `any`, `bool`, `dict`, `enumerate`, `filter`, `float`, `int`, `iter`, `len`, `list`, `map`, `max`, `min`, `pow`, `print`, `range`, `reversed`, `round`, `sorted`, `sum`, `zip`。

1 Is it too difficult? (14pts)

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. Each expression has at least one line of output.

- If an error occurs, write **Error**, but include all output displayed before the error.
- To display a function value, write **Function**. To display an iterator or generator value, write **Iterator**.
- If an expression would take forever to evaluate, write **Forever**.

The interactive interpreter displays the value of a successfully evaluated expression unless it is None.
Assume that you have first started python and executed the statements below.

```
def make_account(balance):
    def account(opcode, amount):
        nonlocal balance
        if opcode == 1:
            balance = balance - amount if amount <= balance else balance
        else:
            balance += amount
        return balance
    return account

withdraw = lambda account, amount: account(1, amount)
top_up = lambda account, amount: account(2, amount)

def series1(start, step):
    yield start
    yield from series1(step(start), step)

def series2(start, step):
    while True:
        yield start
        start = step(start)
```

Expression	Interactive Output
`pow(2, 11) - 24`	`2024`
`print(7, 8, 9) * 6`	`7 8 9` `Error`
`print(print('Hello'), 'SICP')`	
`print(False) and 1 / 0`	

`lst1 = [1, 2, 3, 4]`
`lst2 = lst1 + [5]`
`lst1.append(5)`
 	
`lst1 == lst2`
`lst1 is lst2`	

`alice_account = make_account(10)`
`bob_account = make_account(10)`
 	
`withdraw(alice_account, 20)`
`top_up(alice_account, 10)`
`top_up(bob_account, 10)`
`withdraw(bob_account, 20)`	

`nat = series1(0, lambda x: x + 1)`
`even1 = series2(0, lambda x: x + 2)`
`even2 = map(lambda x: x * 2, nat)`
`even3 = filter(lambda x: x % 2 == 0, nat)`
 	
`[next(nat) for _ in range(5)]`
`[next(even1) for _ in range(5)]`
`[next(even2) for _ in range(5)]`
`[next(even3) for _ in range(5)]`
`list(nat)`
`list(even1)`	

2 Environment Diagram (14pts)

Fill in the environment diagram that results from executing the code on the right until the entire program is finished, an error occurs, or all frames are filled.

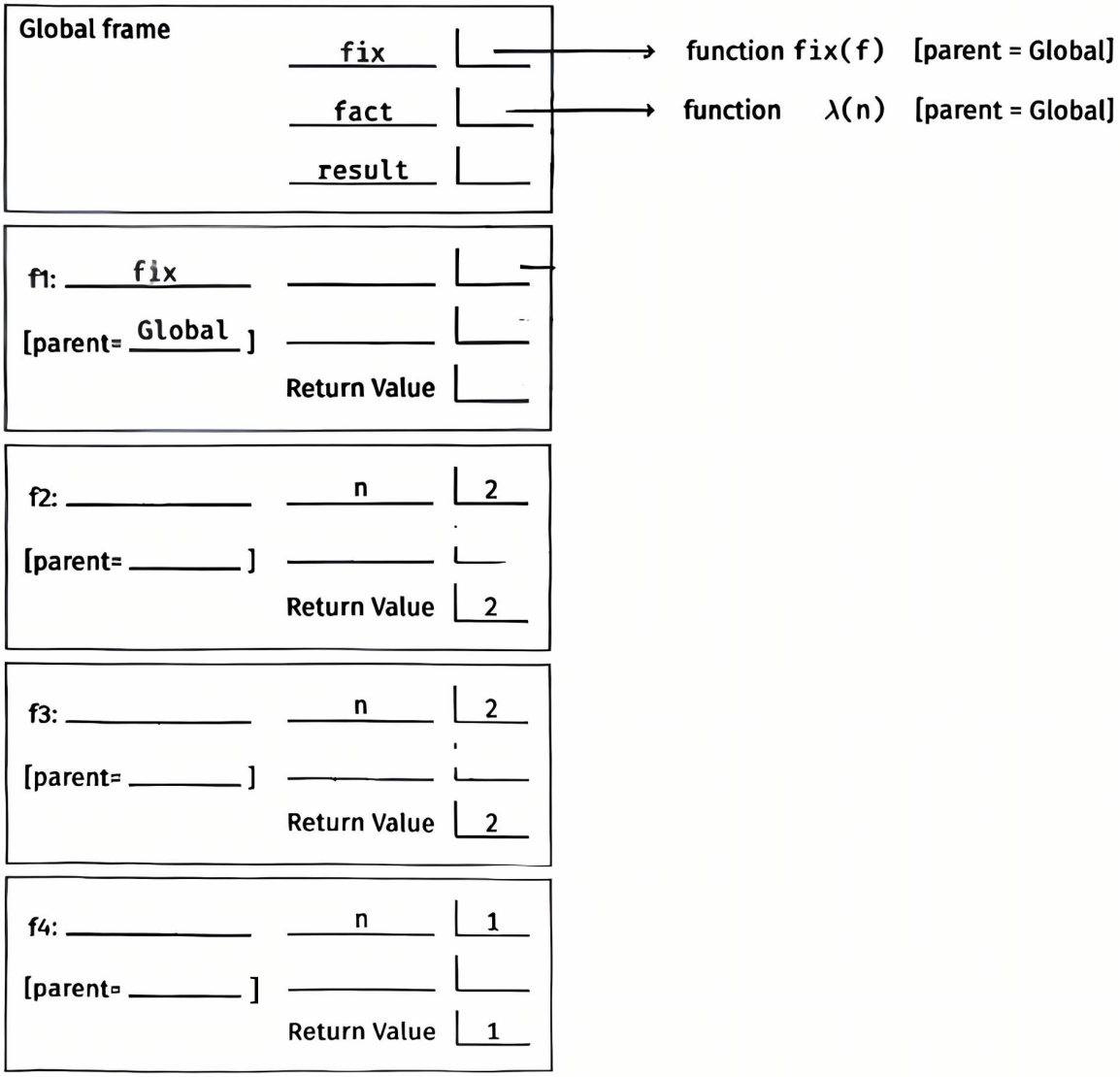
A complete answer will:

- Add missing names and parents to all local frames.
- Add missing values created or referenced.
- Show the return value for each local frame.

Hint:

- **You may not need to use all of the spaces or frames.**
- **If a variable is assigned more than once, draw all of its possible values.**

```
1  def fix(f):
2      def inner(n):
3          return 1 if n <= 1 else f(n)
4      return inner
5
6  fact = lambda n: n * fact(n - 1)
7  fact = fix(fact)
8  result = fact(2)
```



3 Lists (22pts)

Which expressions in the table can get the **unwrapped** cake? Suppose we started python and executed the statements below. For each expression in the table, if evaluating it returns the string "cake", write **T**. Otherwise, write **F**. **Any changes made to the list in the previous row of the table will not affect the next row.** (5pts)

```
1 lst = [["trap"], ["cake"], ["password"]]
2 lst[1] = [lambda x: ["cake"] if x == lst[2] else lst[0]]
3 lst[0].append(lst)
```

Expression	Your Answer
`(lambda: "cake")()`	T
`(lambda x: "cake")()`	F
`lst[1][0]("password")[0]`	
`lst[1][0]("password")[1][1][0](["password"])`	
`lst = [item for sublist in lst for item in sublist]` `lst[1][0][1][1][0](lst[1][0][1][1][0])[0]`	
`lst = lst[::2] + lst[0]` `lst[0][1][1][0](["password"])[0]`	
`lst.append(lst.pop())` `lst[1][0](["password"])[0]`	

Write expressions involving the list `l` that evaluate to the following values.
Each expression shall be **a single list comprehension** involving the list `l`. (4pts)

```
1 >>> l1 = [[1, 9], [2, 8], [3, 7], [4, 6], [5, 5]]
2 >>> l2 = ['Nothinghere', 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i']
3 >>> [x for x, y in l1 if y < 8]  # an example for you
4 [3, 4, 5]
5 >>> _____________________________________________________
6 [[1, 'i'], [2, 'h'], [3, 'g'], [4, 'f'], [5, 'e']]
7 >>> _____________________________________________________
8 [[[2, 8], 'f'], [[4, 6], 'b']]
```

Since many students are studying hard in the SICP course, the TAs want to give some extra credit to some excellent students. However, adding credits one by one is too time-consuming, so we want to write a program that can add credits in bulk quickly. Could you help us complete this program?

Hint: A **prefix sum** of a sequence of numbers is the sum of the first n elements for some positive length n. The **difference sequence** of a sequence is constructed by taking the difference between each element and the one before it; specifically, the i-th element in the difference sequence is the difference between the i-th and (i−1)-th elements of the original sequence. By calculating the prefix sum of the difference sequence, we can reconstruct the original sequence. We can apply some simple modifications to the difference sequence, and then take a prefix sum to achieve our goal quickly.

Implement **difference** and **prefix**. The expected answer is not long and may use list slicing and ranges. (4pts)

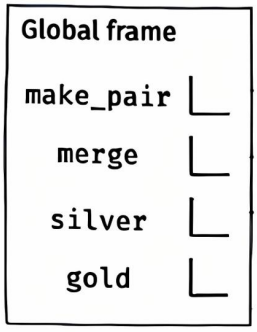
```
def difference(s):
    """Return the difference sequence of list s.
    >>> difference([1, 2, 3, 0, 4, 5])
    [1, 1, 1, -3, 4, 1]
    >>> difference([2, 2, 2, 0, -5, 5])
    [2, 0, 0, -2, -5, 10]
    """
    return ____________________________________________________________

def prefix(s):
    """Return a list of all prefix sums of list s.
    >>> prefix([1, 2, 3, 0, 4, 5])
    [1, 3, 6, 6, 10, 15]
    >>> prefix([2, 2, 2, 0, -5, 5])
    [2, 4, 6, 6, 1, 6]
    """
    return ____________________________________________________________

s = difference([1, 2, 3, 4, 5, 6])
s[0] += 1
s[3] -= 1 # Add 1 credit to the first 3 students
s[4] += 2 # Add 2 credits to the last 2 students
print(prefix(s)) # Will get [2, 3, 4, 4, 7, 8]
```

Draw the final environment diagrams for the following code. Think twice before you write! (9pts)

```
1 make_pair = lambda x, y: [x, y]
2 merge = lambda x, y: make_pair([x[:-1], (x[-1], y[-1])], [y[:-1], (x[-1], y[-1])])
3 silver = make_pair([1, 2, 3], [4, 5, [6, -6]])
4 gold = merge(silver[0], silver[1])
5 silver[0][2] = 9
6 silver[1][2][1] = -9
```



4 Recursion (12pts)

Implement `merge`. `merge` takes two **sorted** lists `xs` and `ys` and return `sorted(xs + ys)`. You may not use `sorted` in your answer. (7pts)

```
def merge(xs, ys):
    """ Merges two sorted lists into a single sorted list.
    >>> merge([1, 2, 3], [0, 4, 5])
    [0, 1, 2, 3, 4, 5]
    >>> merge([1, 2, 3], [2, 3, 4])
    [1, 2, 2, 3, 3, 4]
    """
    if ____________________:
        return ys
    elif ____________________:
        return xs
    elif ____________________:
        return ____________________________________
    else:
        return ____________________________________
```

Implement `merge_sorted` using `merge`. `merge_sorted` takes a list `xs` and return `sorted(xs)`. You may not use `sorted` in your answer. (5pts)

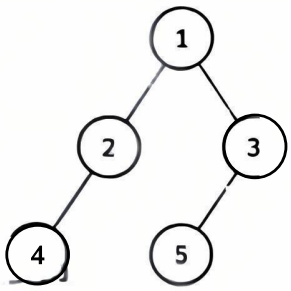
```
def merge_sorted(xs):
    """ Returns the sorted version of xs.
    >>> merge_sorted([3, 7, 6, 1, 2, 0, 4, 5, 9, 8])
    [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
    """
    l = len(xs)
    if ____________________.
        return ____________________
    else:
        return ____________________________________________________________
```

5 Trees (23pts)

5.1 Warming up (10pts)

The following tree APIs are available for this problem:

- `tree(label, branches=[])`
- `label(tree)`
- `branches(tree)`
- `is_leaf(tree)`
- `copy_tree(tree)`
- When printing a tree, you will get a string that, when executed, constructs the same tree.



Define the tree in the figure using `tree` constructor. (3pts)

```
t1 = _____________________________________________________
```

Implement `map_tree`, which takes in a tree `t` and a function `f`. `map_tree` replaces each label `l` from the original tree with a newly computed label mapped by the function `f`, `f(l)`. (4pts)

```
def map_tree(t, f):
    """ Map each label l to f(l), returns the new tree.
    >>> map_tree(t1, lambda x: x * 2) # What you defined in the first question
    tree(2, [tree(4, [tree(8)]), tree(6, [tree(10)])])
    """
    return ____________________________________________
```

Implement `height(t)` using list comprehension. (3pts)

`height` computes the height of a given tree `t`. The height of a tree is inductively defined as follows:

1. The height of a leaf is 1.
2. The height of a tree `t` is the maximum depth of its branches plus one.

```
def height(t):
    """ Returns the height of a tree.
    >>> height(t1)
    3
    """
    if ____________________:
        return 1
    return ____________________________________________
```

5.2 What went wrong? (13pts)

Here is a failed attempt to solve the problem add_trees written by students who enrolled in the course SICP, could you find out what went wrong?

```
1  def add_trees(t1, t2):
2      l1, l2 = label(t1), label(t2)
3      if len(t1) == 1:
4          return [l1 + l2, [copy_tree(b) for b in branches(t2)]] #Deep copy
5      elif is_leaf(t2):
6          return tree(l1 + l2, t1[1:])
7      else:
8          new_branches = []
9          br1, br2 = branches(t1), branches(t2)
10         for i in range(max(len(t1), len(t2))):
11             if i < len(br1):
12                 if i < len(br2):
13                     new_branches.append(add_trees(br1[i], br2[i]))
14                 else:
15                     new_branches.append(copy_tree(t1[i]))
16             else:
17                 new_branches.append(copy_tree(br2[i]))
18         return [l1 + l2] + list(new_branches)
```

You may not need to use all of the blanks below.

Line Number	Correct Answer
3	is_leaf(t1)

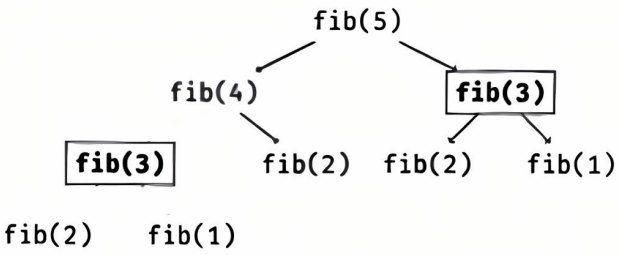
6 Remember Me! (15pts)

6.1 Motivation (3pts)

We, programmers, write recursive functions to solve recursive problems. As an example, to compute the nth number of the classic Fibonacci sequence, we can directly translate the recursive definition of the Fibonacci sequence into a naive recursive function named `fib(n)`.

$$F_n = \begin{cases} 1, & n = 1, 2 \\ F_{n-1} + F_{n-2}, & n \geq 3 \end{cases} \tag{1}$$

```
1 def fib(n):
2     if n == 1 or n == 2:
3         return 1
4     else:
5         return fib(n - 1) + fib(n - 2)
```



However, the function `fib` is not efficient. For example, to evaluate `fib(5)`, python will have to evaluate `fib(3)` twice, one for `fib(4)` and one for `fib(5)`. Suppose `fib(1)` and `fib(2)` return immediately and do not make subsequent calls to `fib` as they are the base cases for computing the Fibonacci sequence, write a function `count_calls(n)` to count the number of function calls to `fib` that is needed to evaluate `fib(n)`. To give an example, `count_calls(5)` returns 9 since python needs to evaluate the function `fib` 9 times to compute `fib(5)`.

```
1 def count_calls(n):
2     if ____________________
3         return 1
4     return ____________________________________________
```

According to `count_calls`, the running time of the function `fib` grows _____ as the input argument n increases.

(A) Constantly
(B) Linearly
(C) Logarithmically
(D) Exponentially

To make our lives easier, we definitely want to speed up the process of computing `fib(n)`. But before digging into details, we will have a closer look at the function `fib` we are going to optimize to identify some useful properties of it. `fib` is a pure function, it is recursive, it has no side effects, it takes exactly one argument, etc. But among these properties, the most important is that it is a so-called *pure* function.

6.2 Pure (4pts)

Simply put, a function is **pure** if it always returns the same value when given the same arguments and has no side effects (i.e., changing some global states). With pure functions, we can remember the function's results for some given inputs. When the function is called next time with the same inputs, we can immediately return what we remembered since the results remain the same, and we need not worry about missing side effects even though we skipped the function's body.

For each of the functions in the table, write **T** if it is a pure function, write **F** otherwise. If the function has an input argument, you can assume that the argument will always be **positive integers**.

Function	Your Answer
`song = lambda : 'APT'`	T

`import random`
`def lottery():`
`  return random.randint(6, 71)`	

`def fact(n):` `  return 1 if n == 0 else n * fact(n - 1)`	

`t = []`
`def append_len(x):`
`  t.append(x)`
`  return len(t)`	

`def fun(x):`
`  def safe(y, total):`
`  nonlocal x`
`  if y <= 0:`
`  return total`
`  x = (y - 1) * 2`
`  return safe(y - 1, x * (total + 1))`
`  return safe(x, 0)`	

6.3 Rewrite Me? (2pts)

a pure function with only one argument n, we can always write another function to do the same thing as the nal function does, except that the new function will *remember* the result Fn for a given n after computation, *recalls* the *remembered* result next time immediately if it has *remembered* the result for that same argument n. atively speaking, we can write a function that *learns* from its past!

complete function `fib_r`, the improved version of `fib`.

```
= memory()
fib_r(n):
if ____________________
    return recall(mem, n)
Fn = 1
if _________________:
    Fn = fib_r(n - 1) + fib_r(n - 2)
remember(mem, n, Fn)
return Fn
```

Here, we use an ADT called `memory` to

- `memory()`
  create a new memory that remembers nothing
- `remember(mem, n, Fn)`
  remember that the nth element of the sequence is Fn, and store this piece of information in memory `mem`.
- `recall(mem, n)`
  return the nth element of the sequence if it is remembered, returns **None** if not.

r seems nice, and surely does a great job in improving the efficiency of `fib`. However, having to write a letely *new* function to solve an already solved task is laborious, as we are lazy programmers and are unwilling write existing code (aren't we all?).

6.4 Remember Me! (6pts)

To go one step further, let's write a function `remember_me(f)` to produce an improved version of *any* given function `f` so that we do not have to write functions over and over again to speed up existing functions.

```
def remember_me(f):
    ________________
    def inner(n):
        if _________________:
            return recall(mem, n)
        ________________________
        ________________________
        return ___________________
    return inner
```

After finishing `remember_me`, we can simply use one assignment statement to bind the variable `fib` to the improved version created by `remember_me`. Later call expressions whose operator is `fib` will then call the efficient remembered version of the original `fib` function. We give an example here to demonstrate the power of `remember_me` by computing the 100-th number of the Fibonacci sequence.

```
>>> fib = remember_me(fib)
>>> fib(100) # This is huge
354224848179261915075
```

Moreover, `remember_me` enables us to transform *any* pure function with one argument into an efficient remembered version, using only one assignment statement (the power of abstraction, again!). And as we will soon see, the restriction on the number of arguments is not really that big a problem since we can do some clever encoding and decoding to trick the Python interpreter.

For pure functions with more than one argument, we can always pack the arguments into a single tuple to mimic a single-argument function. We use the function `count_k` from lab03 as a demonstration.

```
def count_k(n, k):
    """Counts the number of paths to climb up a flight of n stairs,
    taking up to and including k steps at a time."""
    if n == 0:
        return 1
    total, i = 0, 1
    while i <= min(n, k):
        total += count_k(n - i, k)
        i += 1
    return total
```

For count_k, we can rewrite it to accept a single tuple as the input argument.

```
def count_k_1(nk):
    """Counts the number of paths to climb up a flight of n stairs,
    taking up to and including k steps at a time."""
    n, k = nk
    if n == 0:
        return 1
    total, i = 0, 1
    while i <= min(n, k):
        total += count_k_1((n - i, k))
        i += 1
    return total
```

```
>>> count_k_1 = remember_me(count_k_1)
>>> count_k_1((100, 2)) # Also huge
573147844013817084101
```

Again, since we are all lazy programmers, can we write another function, say, `two_to_one(f)`, to transform a pure function f with two arguments n and k into another function that behaves exactly the same, except that the transformed function uses a tuple `(n, k)` to encode the two arguments of the function f?

Here is an attempt to do so. Figure out what would interactive Python interpreter display after evaluating the given expression.

```
def two_to_one(f):
    return lambda nk: f(nk[0], nk[1])
```

```
>>> count_k = two_to_one(count_k)
>>> count_k((5, 3))
_________________
```

Bonus (3pts)

1. The text book of this course is _____ (in English)
2. The full name of this course is _____ (in English)
3. Name three TAs of this course: _____ (in Chinese)